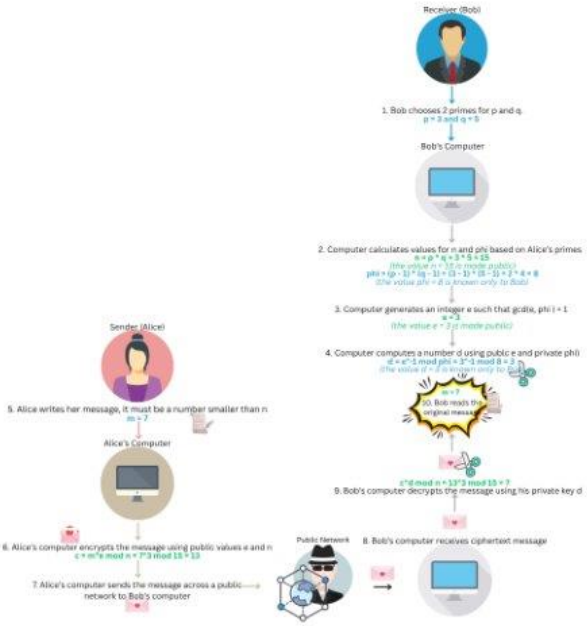
Summer Research 2025 Essay Report

This summer I worked with Dr. Ziliak to research the security of the RSA algorithm. RSA is the most widely used system today to securely send messages from one party to another. When it was first brought to life in 1977, RSA was generally rejected due to it being one of the first public key cryptosystems. The same year, the National Security Agency (NSA) proposed a federal Data Encryption Standard (DES) which implements asymmetric cryptosystem which was more popular at the time. DES also ran much faster since it required a smaller key size than RSA. In the end, a hybrid cryptosystem was made using the speed of DES combined with the security of a public-key system of RSA.

The RSA algorithm starts on the side of the reciever, Bob, who picks two prime values p and q. These values allow us to compute the public key values e and n that can be seen by anybody and the private key values k and d which are only known to Bob. Then, the sender, Alice, sends a message to Bob and using the Bob's public e value, she encrypts it so that no one can understand what it says. She send the message over a public network to Bob. Bob receives the ciphertext message and decrypts it with his private decryption value d. The original message that Alice sent can then be read. The image below depicts the RSA process using an example.

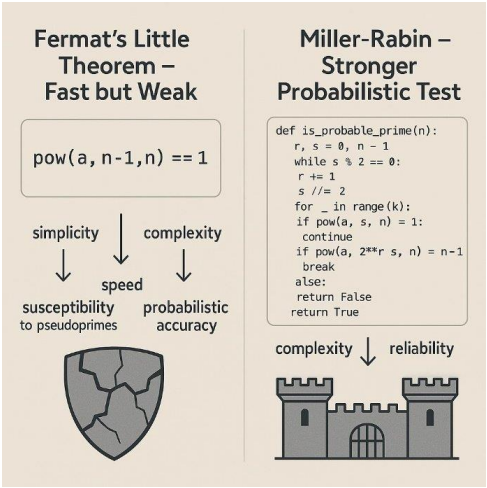


We mimicked the RSA encryption-decryption scheme in our own implementation of the code. Using this code, we performed a cryptanalysis of the system. Cryptanalysis is the process of acting as a hacker and attacking your own code to expose vulnerabilities within it, and then further secure your system based on those found vulnerabilities. We performed 3 attacks on our code this summer: 2 brute force attacks and 1 attack specific to systems that use small private exponent values.

Our first brute force attack looped through all possible message values and ran it through the encryption code. If the resulting ciphertext matched the public encrypted message (c), the brute force attack was successful. When we first ran this code, we found that the brute force attack was able to uncover our message within 0.005 seconds. So, our code was clearly very vulnerable. We found that this was due to the size of our private prime values p and q. Since our

code received those primes from the user's manual input, we were using very small numbers like 3 and 5. To further secure our systems, we needed to increase the size of our prime numbers.

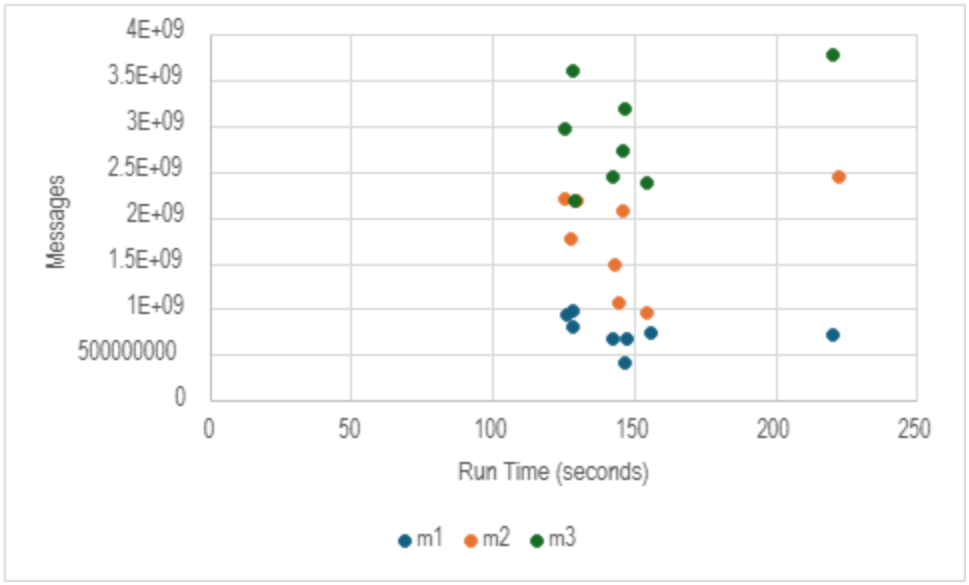
To further secure our system, we increased the size of our primes. When using 16-bit size primes for p and q, we noticed the run time for the brute force attack was still very small. So, we further increased the number of bits to 32 and noticed it took a lot longer for the system to uncover our message using brute force. Seeing the direct relationship between bit-size and system security, we tried to further increase our primes to 64-bit. At this point, the system took so long to break our code, we determined that at 64-bits, our message could not be uncovered with brute force.



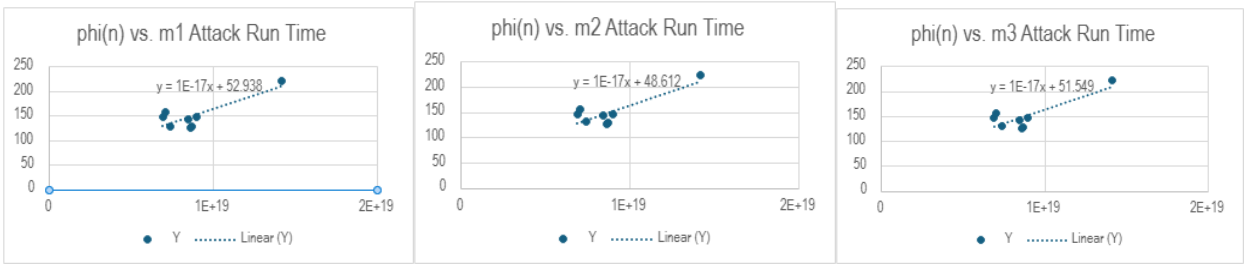
We generated these large primes using an algorithm called the Miller-Rabin Primality Test. This test makes use of Fermat's Little Theorem (FLT) as its foundation to ensure that a generated number is actually prime (and not composite). FLT asserts that if p is a prime number and x is relatively prime to p, then x^(p – 1) =1 mod p. In other words, if we want to test if a number p is prime, we can check it by plugging it into the FLT equation x^(p – 1) and if we get a remainder of 1, the number is prime. If we get any other remainder or no remainder at all, the number is not prime. This test works well in general but has a vulnerability: Carmichael numbers. Carmichael numbers are numbers that pass as prime when plugged into the FLT equation but are not actually prime. For example, let's use x = 2 and p = 561 (a Carmichael number). Then 2^(561 – 1) mod 561 = 1 but, 561 = 3 *11 * 17 so it is clearly composite! To prevent Carmichael numbers from being passed as prime in our test, we combine Fermat's Little

Theorem with a more secure and accurate probabilistic primality test and that is the Miller-Rabin Primality Test.

In the second brute force attack, we wanted to see how each component of the RSA scheme affects the run time of our attack. This attack is meant to be a cleverer version of the first brute force attack. This is because it directly loops through all factors of n until it finds the correct values for p and q. We wanted to see if the size of our message had any effect on how long it would take for the attack to uncover it. Will longer messages take longer to uncover? Then, will shorter messages be easier to uncover? To test this, we had the computer generate 3 messages of different sizes: m1 which ranged from 1 to q/3, m2 which ranged from q/3 to 2q/3, and m3 from 2q/3 to q. We attacked each message and checked how long it took for the attack to successfully uncover them. We found that for all the messages m1, m2, and m3 it took on average 150 seconds for the attack to crack it. This went against our initial hypothesis. In the future, we hope to further investigate why the message size has no effect on this attack.



Similarly, we hypothesized that the size of our private phi(N) value must have some impact on how fast the attack is able to uncover our message. We ran the attack with our 3 messages and found that our initial hypothesis had again been proven wrong. As you can see from the graph below, the run time on each message did not change much at all and thus phi(N) and attack run time don't have any definitive relationship.



Lastly, we ran an attack on systems that use small values for their private exponent d. This value d represents the decryption key in the RSA scheme. It was found that if the d value is small enough, we can mathematically find it through Weiner's Attack using the public key. This attack is based on the RSA equation d = e^-1 mod phi(N). With basic algebra, we can change this equation to become (e/phi(N)) - (k/d) = 1/(d x phi(n)). Weiner states that since d and phi(N)

are both very large numbers, e/phi(N) - (k/d) is equal to something very small. In other words, the difference between the two is very small, so they are really close. According to Legendre's Theorem, we can assume that phi(N) is approximately equal to N since phi(N) = (p – 1) * (q – 1) = (p x q) - (p + q) + 1 = N - (p + q) + 1. This sets the stage for Weiner's attack. Since phi(N) is almost the same as N, we can replace phi(N) with N. Combine this with the fact that e/phi(N) is very close to k/d and we get the e/N = k/d which is the basis of Weiner's Attack. From here, Weiner's Attack expands e/N into a continued fraction. Out of all the convergents of the continued fraction, one of them will be k/d, our decryption key, according to Weiner.

Let’s walk through an example of how the attack works. We're pretending to be a hacker, so from our perspective we only know the public key (e, n). Let's start with those values: e/N = 42667/64741.

1. Let's expand this into a continued fraction using the Euclidean algorithm:

→ 42667 ÷ 64741 = 0 remainder 42667

→ 64741 ÷ 42667 = 1 remainder 22074

→ 42667 ÷ 22074 = 1 remainder 20593

→ 22074 ÷ 20593 = 1 remainder 1481

→ 20593 ÷ 1481 = 1 remainder 1340

→ 1481 ÷ 1340 = 1 remainder 141

→ 1340 ÷ 141 = 9 remainder 71

→ 141 ÷ 71 = 1 remainder 70

→ 71 ÷ 70 = 1 remainder 1

→ 70 ÷ 1 = 70 remainder 0

2. Now, we can calculate the convergents according to the following formula: hn = an* h(n − 1) + h(n−2) and kn = an k(n−1) +k(n−2)

→ H0 = 0 and k0 = 1

→ H1 = 1 * 0 + 1 = 1 and k1 = 1 * 1 = 1

→ H2 = 1 * 1 + 0 = 1 and k2 = 1 * 1 + 1 = 2

→ H3 = 1 * 1 + 1 = 2 and k3 = 1 * 2 + 1 = 2

→ H4 = 13 * 2 + 1 = 27 and k4 = 13 * 3 + 2 = 41

→ H5 = 1 * 27 + 2 = 29 and k5 = 1 * 41 + 3 = 44

→ H6 = 9 * 29 + 27 = 288 and k6 = 9 * 44 + 41 = 437

→ H7 = 1 * 288 + 29 = 317 and h7 = 1 * 437 + 44

→ H8 = 1 * 317 + 288 = 605 and k8 = 1 * 481 + 437 = 918

→ H9 = 70 * 605 + 317 = 42667 and k9 = 70 * 918 + 481 = 64741

So, our convergents are: [0/1, ½, 1/3, 2/3, 27/41, 29/44, 288/437, 317/481, 605/918, 42667/64741].

3. Next, we will run these convergents through Weiner's 3 tests. If a number passes all 3 tests, we can assume that it represents our private key k/d.

- Test 1: Check if the equation phi(N) = ((e x d) – 1) / k returns a whole integer value.

  → phi(n) = ((42667 x 1) - 1)/0 = undefined FAIL

  → phi(n) = ((42667 x 2) - 1)/1 = 85333 PASS

  → phi(n) = ((42667 x 3) - 1)/1 = 128000 PASS

  → phi(n) = ((42667 x 3) - 1)/2 = 64000 PASS

  → phi(n) = ((141 x 41) - 1)/27 = 214.074074 FAIL

→ phi(n) = ((42667 x 44) - 1)/27 = 69531.3704 FAIL

→ phi(n) = ((42667 x 437) - 1)/288 = 64741.2431 FAIL

→ phi(n) = ((42667 x 481) - 1)/317 = 64740.776 FAIL

→ phi(n) = ((42667 x 918) - 1)/605 = 64741 PASS

→ phi(n) = ((42667 x 64741) - 1)/42267 = 65353.686 FAIL

Since p and q are whole integers, phi(N) should also be a whole integer

- Test 2: Discard all convergents that fail at step 1 and check if the equation p + q = N − phi(N) + 1 returns a positive value.

  → p + q = 64741 – 85333 + 1 = -20591 FAIL

  → p + q = 64741 – 128000 + 1 = -63258 FAIL

  → p + q = 64741 – 64000 + 1 = 742 PASS

  → p + q = 64741 – 64741 + 1 = 1 PASS

  Since p and q are positive values, p + q should also be a positive value

- Test 3: Discard all convergents that fail in step 2. Find the discriminant of the quadratic equation x^2 - (p q) * x +N = 0 using the quadratic formula.

  → x^2 – 742x + 64741 = 0

  Plug equation into the Quadratic Formula:

  → x = [742 ± sqrt((742)^2 – 4 * 1 * 64741)] / 2 * 1

  → x [742 ± 540]/2

  → x = (742 + 540)/2 and x = (742 – 540)/2

  → x = 641 and x = 101

So, we can conclude that our private key k/d = 2/3 which means our decryption key d is 3!

Using this attack, we investigated different components of RSA and their relationship with the attack run time. We found that this attack can uncover messages a lot faster than our brute force attacks, even while using large primes. Switching from 16-bit primes to 32-bit primes to even up to 64-bit primes had little effect on the run time. So again, the size of our message did not influence how long it took for the attack to uncover it. The graph shown below further emphasizes this.

Additionally, we wanted to test the relationship between our public value encryption value e and our private decryption value d. We collected data on the two RSA values, but found that although the two are related, the relationship is not significant enough to have much of an influence over the run time of the Weiner's attack.

In the future, we hope to further investigate modifications made to RSA that go beyond the realm of modular arithmetic. With the rise of quantum computers and the threat they pose to current systems, we look to create a system to secure our future.

Works Cited

Burger, E. B., & Starbird, M. (2013). Public-key cryptography. In The heart of mathematics: An invitation to effective thinking (4th ed., pp. 687–704). Wiley.

Diffie, W., & Hellman, M. (1976). New directions in cryptography [PDF]. Stanford University. https://www.cs.virginia.edu/~evans/greatworks/diffie.pdf

Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126. https://people.csail.mit.edu/rivest/Rsapaper.pdf

Boneh, D. (n.d.). Twenty years of attacks on the RSA cryptosystem [PDF]. Stanford University. https://crypto.stanford.edu/~dabo/papers/RSA-survey.pdf

Narayanan, A. (2014). RSA: Mathematics and Attacks [PDF]. MIT PRIMES.

Packet Mania. (2023, November 17). RSA attack & defense #2 – low private exponent attack. https://www.packetmania.net/en/2023/11/17/RSA-attack-defense-2/#low-private-exponent-attack

Computerphile. (2017, March 23). How RSA Works (Public Key Cryptography) [Video]. YouTube. https://youtu.be/OpPrrndyYNU

Microsoft Copilot. (n.d.). Chat about RSA cryptography. https://copilot.microsoft.com/chats/jG64gahZzSqQV5Gqw4SGX

Ziliak, E. (n.d.). Public-Key Cryptography [PDF]. Benedictine University Cybersecurity Summer Research Project. https://benil.sharepoint.com/teams/CybersecuritySummerResearchProject/Shared%20Documents/General/Papers/Reading%20Discussion%201-%20Public-Key%20Cryptography.pdf